

CEG5201

Hardware Technologies, Principles, & Platforms

Group 4 CA2 Report

Name	Matriculation number
ZHOU JUNMIN	A0257966H
RICARDO JACK LIWONGAN	A0353688L
ZHANG FANGYU	A0353200Y
DIVYALAKSHMI BALASUBRAMANIAM	A0241265L

Semester I, AY2026/27

1 Abstract

This report benchmarks Strassen, Coppersmith–Winograd (CW), Cannon and Fox matrix multiplication in Python, each implemented sequentially and with multiprocessing.Pool using hand-written integer loops. Ten groups of eight random integer matrix pairs of order 16 to 2048 were run on one Intel Core i5-12500H with one to six worker processes. Strassen and CW replace eight half-size block products with seven and send the recursive sub-products to the workers; Cannon and Fox use a fixed 4 × 4 block grid with one task per output block. Strassen has the shortest sequential total, 2639 s, and CW the shortest six-worker total, 751 s. Over all groups, six workers give speed-ups of 3.26 to 3.56, an efficiency of 54–59%. With only 16 tasks, Cannon and Fox gain nothing from a fifth worker, and every algorithm is slower through the pool for 16 × 16 matrices. Useful parallelism therefore depends on the number and size of the tasks and on data movement, not on the process count alone.

2 Introduction

Matrix multiplication is a basic operation in scientific computing, graphics and neural-network processing. Its regular structure offers substantial parallelism. Despite this, transferring inputs, coordinating workers and assembling results introduce overhead that can limit speedup. This project investigates whether dividing the work among several CPU processes reduces execution time and how the benefit differs between algorithms.

The technical problem is to compute Z=XY for square matrices of order M, where each entry is $Z[i,j] = \Sigma_{k=0}^{M-1} X[i,k]Y[k,j]$. Four algorithms are implemented sequentially and using Python multiprocessing. Strassen replaces eight half-size products with seven, at the cost of additional block additions and subtractions [1], [2]. Coppersmith–Winograd follows the seven-product Strassen–Winograd variant in Appendix A, rather than the full asymptotic construction of the 1990 paper [1], [3], [4]. Cannon uses initial block alignment followed by cyclic shifts, while Fox uses staged block selection corresponding to broadcast-and-roll operations. Both organise conventional block products over a fixed 4 × 4 logical grid.

All implementations perform arithmetic using explicit Python loops, with NumPy for matrix storage and separate correctness checks. Strassen and Strassen–Winograd switch to conventional multiplication for small subproblems. Their multiprocessing versions distribute recursive subproblems, while Cannon and Fox assign output-block computations to workers. Parallel execution uses standard-library process pools with the spawn start method and one to six worker processes [5].

The final evaluation runs on Windows 11 using an Intel Core i5-12500H with 12 physical cores, 16 logical processors and 15.7 GiB of reported RAM, with Python 3.14.7 and NumPy 2.5.3. A shared framework supplies identical matrix inputs, execution order and timing boundaries for all algorithms. Speedup is calculated from sequential and multiprocessing elapsed times and interpreted through Amdahl's law, task granularity and communication overhead [6], [7].

3 Matrix generation

The input set contains ten groups, G0-G9, each comprising eight independent pairs (A_i, B_i). The matrix orders are 16, 32, 64, 128, 256, 512, 1024 and 2048. The full benchmark therefore contains 80 pairs and 160 matrices. The generation code in common/data.py uses NumPy's default_rng([5201, g]), where 5201 is the fixed seed and g is the group index. Matrix entries are sampled uniformly from the integers 0–255, inclusive. Each file, G0.npz–G9.npz, stores eight pairs labelled A0/B0–A7/B7 in uint8 format. The .npz format stores multiple binary

arrays in a single archive [8]. Together, the ten files occupy 111,884,380 bytes. No external dataset is used. Every saved matrix was checked for the required shape, integer type and value range. The data fingerprint, b7c06b8479fd4602, comprises the first sixteen hexadecimal characters of a SHA-256 digest calculated over the arrays in group and pair order. It matches all 28 final Windows run records, supporting consistency of the benchmark inputs.

4 Algorithm implementation

The timing framework uses time.perf_counter() to record cumulative elapsed time after each matrix pair. Loading, conversion to int64, garbage collection and pool startup occur before timing; a barrier ensures all workers are ready. Timed work includes algorithm-specific preparation, arithmetic, inter-process transfers and result assembly. All five specified BLAS/OpenMP thread limits are set to one before importing NumPy; no shared memory is used. Correctness is checked separately using verify.py. The script profile_seq.py uses cProfile to report function call counts, self-time and cumulative time.

4.1 Strassen

Strassen divides each input into four quadrants and replaces the usual eight half-size products with seven. The implementation follows the exact M_1–M_7 and reconstruction formulas in Appendix A [1], [2]. For example, $M_3 = (X_{11} - X_{22})(Y_{11} + Y_{22})$, and $Z_{11} = M_2 + M_3 - M_6 - M_7$. Extra terms cancel when the products are combined. Each level uses seven recursive products and 18 block additions or subtractions, giving $O(M^{\log_2 7})$, approximately $O(M^{2.807})$, arithmetic complexity.

In algorithms/strassen.py, multiply_seq converts the int64 input arrays to lists inside the timed call. All subsequent arithmetic uses Python integers, including negative intermediate values. _multiply stops at STRASSEN_LEAF_SIZE = 64, or an odd order, and calls _base_multiply. This is the required i-j-k loop; Y is stored by columns once per leaf. _add and _sub use scalar loops, while _split and _combine build and join the quadrants. The assigned powers of two need no padding.

The repeated leaf products are the main computational work: at order 2048, five recursion levels give $7^5 \times 64^3 = 4.41$ billion scalar multiplications. To distribute this work, multiply_mp prepares independent operand pairs in the main process until there are at least 7p pairs or the cutoff prevents further splitting. For p = 2–6 this gives seven tasks at order 128 and 49 from order 256. Each apply_async task runs the remaining sequential recursion; workers create no nested pools [5].

The main process collects products in submission order, combines each family of seven while other tasks may still run, then rebuilds the remaining levels. With p = 1, or order at most 64, one whole product is still sent through the pool. List conversion, operand preparation, serialisation and reconstruction remain inside the common timer. Pool startup is excluded. Preparing and copying more blocks offsets some parallel gains [7].

4.2 Coppersmith-Winograd

Following Appendix A [1], the algorithm divides X, Y and Z into four quadrants of order M/2 and forms seven quadrant products instead of eight. One level computes:

$S_1=X_{21}+X_{22},\ S_2=S_1-X_{11},\ S_3=X_{11}-X_{21},\ S_4=X_{12}-S_2,$
$T_1=Y_{12}-Y_{11},\ T_2=Y_{22}-T_1,\ T_3=Y_{22}-Y_{12},\ T_4=T_2-Y_{21},$
$M_1=X_{11}\cdot Y_{11},\ M_2=X_{12}\cdot Y_{21},\ M_3=S_4\cdot Y_{22},\ M_4=X_{22}\cdot T_4,$
$M_5=S_1\cdot T_1,\ M_6=S_2\cdot T_2,\ M_7=S_3\cdot T_3,$
$U_2=M_1+M_6,\ U_3=U_2+M_7,\ U_4=U_2+M_5,$
$Z_{11}=M_1+M_2,\ Z_{12}=U_4+M_3,\ Z_{21}=U_3-M_4,\ Z_{22}=U_3+M_5.$

In algorithms/coppersmith_winograd.py, multiply_seq converts the inputs once with tolist() inside the timed function, so all arithmetic runs on Python integers. _multiply recurses until the order is at most CW_LEAF_SIZE = 64, or odd, and then calls _base_multiply, the required triple loop, which stores the columns of each leaf block of Y once so the inner loop walks a row of X and a column of Y side by side. _operands builds the eight S and T sums, _combine applies the U and Z additions and _join concatenates the quadrants. All assigned orders are powers of two, so no padding is needed. The leaf size of 64 was chosen by measurement, because below it the 15 additions of a level cost more than the product they save. A 1024-order product performs $7^4 = 2401$ leaf products instead of $16^3 = 4096$, 41% fewer. Profiling on the separate AMD Ryzen 7 PRO 7840U diagnostic machine shows _base_multiply takes 89.6% and 88.0% of the run time at orders 512 and 1024, against 5.9% and 6.9% for _add and _sub together [9], so the leaf products are the work to parallelise. They are independent, because each reads only the S and T sums prepared before it [7].

multiply_mp performs the S and T additions of the top CW_MP_DEPTH = 2 levels in the main process, which for $M \geq 256$ gives $7 \times 7 = 49$ independent operand pairs of order M/4, each submitted with apply_async. A free worker runs the sequential recursion on its pair, so all deeper additions and leaf products also run in parallel, and returns the product. Workers create no nested pools and share no memory, so every operand pair and result is pickled between processes. At M = 128 one level gives seven tasks, and orders up to 64 are sent as one task. Because 49 tasks rarely divide evenly, from M = 512 the last 49 mod p pairs are split once more into seven smaller products each, so with six workers 48 large tasks and seven small ones let all workers share the final round instead of one task running alone. Results are collected in submission order, and the main process combines each finished group of seven products while the workers continue. In alternating runs on that diagnostic machine the tail split together with this early combining cut the 2048-order time at p = 6 by about 9%, while at M = 1024 the change was within noise [9]. Depths one to three performed within noise at M = 1024, and depth two was kept because every extra level multiplies the main-process additions and the transferred data by 7/4. The list conversion, the top two levels of additions and the final combine stay serial in the main process, so they bound the speed up that more workers can give.

List conversion, the top-level additions, task submission, transfer, arithmetic and combining all lie inside the common timer, while loading, int64 conversion, garbage collection and pool startup are excluded. Mean pool startup is about 0.27–0.41 s across process counts. verify.py checks both versions for p = 1 to 6 against NumPy on the G0 pairs up to order 256, and all checks pass.

4.3 Cannon

Cannon's algorithm arranges the block product on a q × q grid in which every position holds one block of X and one block of Y, so that each step multiplies blocks that are already in place [10]. Following the revised statement, q = 4: X, Y and Z are divided into 16 blocks of order M/4 [1]. An initial alignment shifts block row i of X left by i positions and block column j of Y up by j positions, so position (i, j) holds X_ik and Y_kj with k = (i + j) mod 4. Each of the four stages then multiplies the pair held at every position and adds the product to Z_ij, after which all X blocks move one position left and all Y blocks one position up, circularly. At stage s the pair has k = (i + j + s) mod 4, so every position meets each k exactly once and $Z_{ij} = \Sigma_k X_{ik} Y_{kj}$. With 0-based indices, for example, position (1, 2) multiplies $X_{13}Y_{32}$, $X_{10}Y_{02}$, $X_{11}Y_{12}$ and $X_{12}Y_{22}$ in stages 0 to 3.

The method performs the same M^3 scalar multiply-adds as the conventional product: four stages of 16 block products of order M/4. In every stage each block of X and of Y sits at exactly one grid position, and the stages only move blocks between neighbouring positions.

In my code, multiply_seq cuts the NumPy inputs into 4 × 4 views at the boundaries tM/4 and converts each block once with tolist() inside the timed function, so the arithmetic runs on Python integers. Blocks of Y are stored transposed, so that the inner loop walks a row of X and a column of Y side by side. The alignment and the shifts are carried out literally by rotating the 4 × 4 grids of block references, which moves no data. Every block product is computed by _multiply_add, an i-j-k triple loop that accumulates directly into the output block. _join finally concatenates the 16 output blocks. All assigned orders divide by four, so no padding is needed.

Profiling multiply_seq on the development machine shows that _multiply_add takes 99.7% of the run time at order 512 and 99.9% at order 1024 [9], while alignment, shifts, conversion and joining together take less than 0.3%. The block products are therefore the part to parallelise, and they are independent: within a stage the 16 products belong to different positions, and the four products of one position only add into that position's own Zij.

multiply_mp turns each grid position into one pool task, which plays the role of one processor of the grid; the pool runs these 16 virtual processors on its p real workers. For position (i, j) the main process lists the four block pairs that the alignment and the shifts would bring there, in stage order. The worker converts them to lists, runs the four multiply-add stages and returns Zij, and the main process joins the 16 returned blocks. Because each task already holds the blocks of all its stages, the shifts reduce to index arithmetic and no synchronisation is needed between stages. A per-stage design, with one starmap call per stage, was rejected because it adds three barriers and moves the accumulation into the main process.

The 16 tasks are submitted with starmap and chunksize=1, so a free worker always takes the next task. Without shared memory every block is copied to the four tasks that use it: one 2048-order product sends 268 MB of int64 data, four times the size of its inputs.

4.4 Fox

Fox’s algorithm partitions X, Y, and Z into a q × q grid of square blocks and computes Z = XY over q stages [11]. Fox et al. describe a broadcast–multiply–roll procedure that broadcasts selected X blocks across processor rows and cyclically shifts Y blocks upwards. This implementation follows the equivalent orientation shown in the assignment where selected Y blocks are broadcast down block columns, while X blocks shift cyclically left.

Given q = 4, there are 16 blocks per matrix, each of size (N/4) × (N/4). Blocks are stored in row-major order, so block (r, c) has a list index 4r + c. At each stage $s \in \{0, 1, 2, 3\}$, the shared block index is k = (col_b + s) % 4. The implementation selects block X[4 × row_a + k] and

block Y[4 × k + col_b], multiplies them, and accumulates the result into blocks Z[4 × row_a + col_b]. Index selection recreates the broadcast and cyclic shift without physically rearranging the blocks. Each block product and its accumulation are computed using explicitly written Python loops. The sequential version performs 16 block multiplications per stage, giving 64 block multiplications across four stages, before assembling the final matrix.

The saved order-512 profile [9] revealed that the fox.multiply() function accounted for 99.2% of the total execution time, consuming 7.030 seconds across 64 function calls. These calls correspond to the 64 block multiplications. In comparison, block splitting and assembly contributed only a small fraction of the total execution time. These findings establish block matrix multiplication as the primary computational bottleneck, indicating that optimising block management would yield limited performance improvements. Consequently, the multiprocessing implementation focuses on distributing the independent output-block computations across six worker processes to exploit parallelism and reduce overall execution time.

In multiprocessing, the bottleneck is addressed by distributing the block multiplications across multiple processes. The multiprocessing implementation uses Python's multiprocessing.Pool with p = 1–6 worker processes. The 16 output blocks of Z form 16 independent tasks, allowing each worker to handle multiple tasks. Each task receives four X blocks from the corresponding block row and four Y blocks from the corresponding block column, then performs all four stage multiplications and accumulates them locally. Grouping the stages within each task avoids returning intermediate products or synchronising workers between stages. Each task returns its completed block and output index; the main process places the blocks in their correct positions and assembles Z. Input preparation and final assembly remain sequential, while task data transfer and result collection introduce overhead that limits acceleration.

5 Results

All results below use the full Windows dataset, G0-G9 [9]. Times are in seconds.

5.1 Strassen

Sequential results

Table B-1.1 Sequential time of G0 (s)

i	M	Cumulative T_i	Derived t_i
0	16	0.0002006	0.0002006
1	32	0.001385	0.001184
2	64	0.01168	0.01030
3	128	0.09728	0.08560
4	256	0.7411	0.6438
5	512	5.291	4.549
6	1024	37.14	31.85
7	2048	262.0	224.8

Table B-1.2 Sequential time of all ten groups (s)

g	Group time $T(G_g)$	Cumulative through G_g
0	262.0	262.0
1	264.0	526.0
2	264.0	790.0
3	263.4	1053
4	264.5	1318
5	264.0	1582
6	264.2	1846
7	265.1	2111
8	264.4	2376
9	263.8	2639

Table B-1.1 gives cumulative times and pair times derived by subtraction. The 2048-order pair takes 224.84 s, 85.82% of G0. Its time is 7.06 times that of order 1024, close to the sevenfold increase in leaf products. Table B-1.2 gives a total of 2639.43 s; individual groups take 261.98–265.12 s, a spread of 1.2%. Times are rounded to four significant figures; all differences and speed-ups use the unrounded records [9].

Multiprocessing results

Table C-1.1 Cumulative and derived multiprocessing times of G0 (s)

	Measured cumulative MP time $T_i^{(p)}$						Derived MP time for the pair $t_i^{(p)}$					
i	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	0.0008041	0.0004434	0.0004690	0.0005493	0.0005034	0.0004745	0.0008041	0.0004434	0.0004690	0.0005493	0.0005034	0.0004745
1	0.002392	0.002127	0.002328	0.002336	0.002297	0.002305	0.001588	0.001683	0.001858	0.001786	0.001793	0.001831
2	0.01238	0.01481	0.01246	0.01276	0.01408	0.01601	0.009983	0.01268	0.01013	0.01042	0.01178	0.01371
3	0.1022	0.07306	0.05856	0.05136	0.05538	0.05510	0.08982	0.05825	0.04610	0.03860	0.04130	0.03909
4	0.7838	0.4652	0.3484	0.3103	0.2969	0.2764	0.6816	0.3922	0.2898	0.2589	0.2416	0.2213
5	5.411	3.240	2.480	2.085	1.871	1.715	4.627	2.774	2.131	1.775	1.575	1.439
6	37.68	21.91	16.57	13.97	12.27	11.26	32.27	18.68	14.09	11.89	10.39	9.541
7	266.9	153.4	114.2	96.10	83.49	77.03	229.3	131.5	97.62	82.13	71.23	65.77

Table C-1.1 shows that orders up to 64 gain nothing from extra workers because each product is one task. At order 128, four workers give the shortest pair time, 0.03860 s, against 0.03909 s with six; from order 256, six workers are fastest.

Table C-1.2 Group and cumulative multiprocessing times of all ten groups (s)

	Measured MP time for the group $T^{(p)}(G_g)$						Cumulative MP time through the group					
g	**p = 1**	**p = 2**	**p = 3**	**p = 4**	**p = 5**	**p = 6**	**p = 1**	**p = 2**	**p = 3**	**p = 4**	**p = 5**	**p = 6**
0	266.9	153.4	114.2	96.10	83.49	77.03	266.9	153.4	114.2	96.10	83.49	77.03
1	266.5	152.6	113.3	96.42	83.92	77.19	533.4	306.0	227.4	192.5	167.4	154.2
2	266.6	153.9	112.7	96.11	83.69	77.07	800.0	460.0	340.2	288.6	251.1	231.3
3	266.5	153.7	112.9	96.63	83.57	77.42	1066	613.6	453.1	385.3	334.7	308.7
4	267.3	153.0	112.8	96.54	83.83	76.97	1334	766.6	565.8	481.8	418.5	385.7
5	265.7	153.7	112.7	96.65	84.08	77.36	1599	920.3	678.5	578.5	502.6	463.0
6	266.6	153.2	112.5	96.59	83.87	77.19	1866	1073	791.0	675.0	586.5	540.2
7	265.5	153.4	112.8	96.67	83.88	77.01	2132	1227	903.8	771.7	670.3	617.2
8	268.1	155.0	112.8	97.02	83.76	76.95	2400	1382	1017	868.7	754.1	694.2
9	267.2	153.0	112.5	96.46	83.87	76.87	2667	1535	1129	965.2	838.0	771.1

Over all ten groups, Table C-1.2 falls from 2666.82 s with one worker to 771.06 s with six. The one-worker pool costs 1.04% more time than sequential execution. Adding the sixth worker reduces the five-worker total by 7.99%.

Speed-up

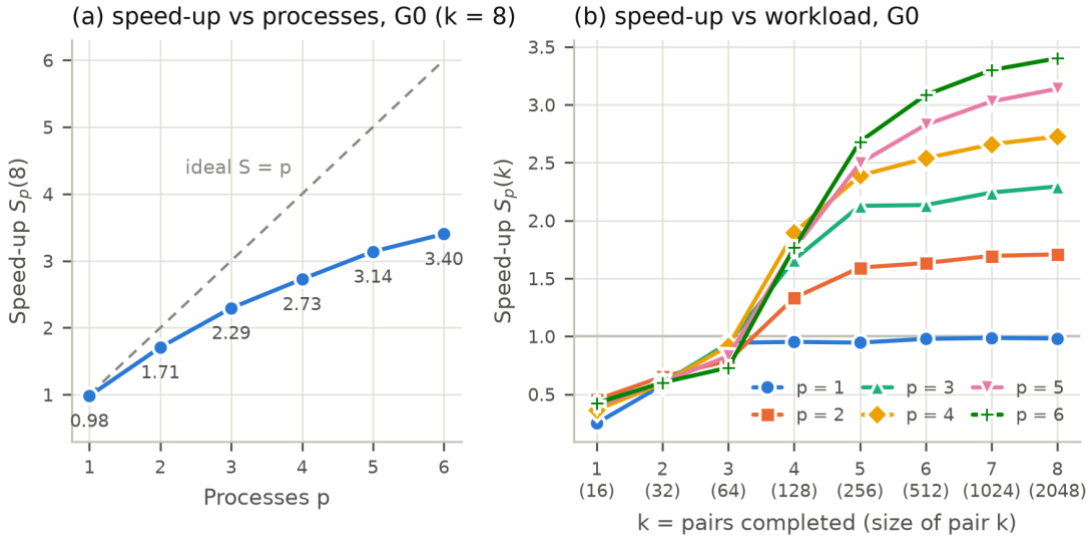


Figure D-1.1 G0 speed-up: (a) versus process count for all eight pairs; (b) versus completed pairs k for each p.

For k completed pairs, Sp(k) is the sequential cumulative time divided by the multiprocessing cumulative time through pair k − 1. Figure D-1.1(a) rises from 0.981 with one worker to 3.401 with six, below the ideal line. In Figure D-1.1(b), p = 2–6 first exceed one at k = 4 (order 128). Six workers become best at k = 5 (order 256), where the task count rises to 49. Their cumulative speed-up then grows from 2.68 to 3.40 as larger products make the arithmetic outweigh task overhead.

Table D-1.2 Speed-up from cumulative times through each group

g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	0.981	1.707	2.294	2.726	3.138	3.401
1	0.986	1.719	2.313	2.732	3.142	3.411
2	0.987	1.717	2.322	2.737	3.146	3.416
3	0.988	1.717	2.325	2.734	3.147	3.412
4	0.988	1.719	2.329	2.735	3.149	3.417
5	0.989	1.719	2.332	2.735	3.148	3.416
6	0.989	1.720	2.334	2.735	3.148	3.417
7	0.991	1.721	2.336	2.736	3.150	3.421
8	0.990	1.719	2.337	2.735	3.150	3.422
9	0.990	1.720	2.338	2.735	3.150	3.423

The first row of Table D-1.2 equals Sp(8) for G0. The final row gives speed-ups of 0.990, 1.720, 2.338, 2.735, 3.150 and 3.423 for p = 1–6. The six-worker result changes by less than 1% as groups are added, so G0 represents the full workload closely. The full-workload efficiency is 3.423/6 = 57.1%.

Bottlenecks

Task rounds. Seven equal tasks at order 128 need two rounds on four to six workers, so extra workers need not help. For larger orders, 49 tasks need nine rounds on six workers, leaving only one task in the final idealised round. The compute-only bound is therefore 49/9 = 5.44, assuming equal tasks and unchanged per-task speed. The measured 3.42 is only 62.9% of that bound; uneven task rounds explain part of the loss, but not all of it.

Serial work and transfer. Input conversion, top-level preparation and reconstruction remain in the parent. Pickling and copying operands and results add costs that cannot be divided by p [6], [7]. Pool startup averages 0.27–0.41 s but is excluded, so it cannot explain the small-matrix slowdown. Shared cache, memory bandwidth, power limits and the mixed core types may also limit acceleration. No saved Strassen task-level profile separates these effects, so their individual contributions cannot be quantified. The ten groups confirm a consistent benefit from six workers, but do not measure variation from rerunning identical inputs.

5.2 Coppersmith-Winograd

Sequential results

Table B-2.1 Processing time of G_0 under sequential implementation (s)

i	M	Cumulative T_i	Derived t_i
0	16	0.0002080	0.0002080
1	32	0.001407	0.001199
2	64	0.01080	0.009390
3	128	0.09576	0.08496
4	256	0.7380	0.6422
5	512	5.394	4.656
6	1024	38.13	32.74
7	2048	266.4	228.3

Table B-2.2 Sequential processing of all ten groups (s)

g	Group time $T(G_g)$	Cumulative through G_g
0	266.4	266.4
1	267.6	534.0
2	274.3	808.3
3	265.9	1074
4	265.5	1340
5	266.2	1606
6	266.8	1873
7	266.6	2139
8	266.6	2406
9	266.3	2672

The 2048-order pair takes 228.3 s, 85.7% of G0. One more recursion level multiplies the number of leaf products by seven, so doubling the order should multiply the pair time by about seven rather than eight. The measured ratios are 7.25 from order 256 to 512, 7.03 from 512 to 1024 and 6.97 from 1024 to 2048, which matches the $O(M^{2.807})$ growth. The ten groups take 265.5 to 274.3 s each and 2672 s in total. Without G2 the spread is 0.8%, and the code does not skip work according to entry values, so the higher G2 total most likely reflects a temporary slowdown of the machine rather than extra work.

Multiprocessing results

Table C-2.1 Processing time of G0 under multiprocessing implementation (s)

	Measured cumulative MP time $T_i^{(p)}$						Derived MP time for the pair $t_i^{(p)}$					
i	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	0.000800	0.000431	0.000439	0.000472	0.000574	0.000483	0.000800	0.000431	0.000439	0.000472	0.000574	0.000483
1	0.002333	0.002069	0.002113	0.002382	0.002454	0.002404	0.001533	0.001638	0.001674	0.001910	0.001880	0.001921
2	0.01261	0.01312	0.01236	0.01249	0.01330	0.01302	0.01027	0.01105	0.01025	0.01011	0.01085	0.01062
3	0.09999	0.07379	0.06011	0.04811	0.05232	0.04799	0.08738	0.06067	0.04775	0.03562	0.03902	0.03497
4	0.7719	0.4729	0.3792	0.3085	0.2939	0.2581	0.6719	0.3991	0.3191	0.2604	0.2416	0.2101
5	5.531	3.236	2.433	2.095	1.872	1.636	4.759	2.763	2.054	1.786	1.578	1.378
6	38.57	21.89	16.50	14.00	12.35	11.10	33.04	18.65	14.06	11.90	10.48	9.464
7	267.9	150.7	113.7	95.02	83.64	74.83	229.4	128.9	97.22	81.03	71.29	63.73

Table C-2.2 Multiprocessing time of all ten groups (s)

	Measured MP time for the group $T^{(p)}(G_g)$						Cumulative MP time through the group					
g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	267.9	150.7	113.7	95.02	83.64	74.83	267.9	150.7	113.7	95.02	83.64	74.83

	Measured MP time for the group $T^{(p)}(G_g)$						Cumulative MP time through the group					
g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
1	267.3	151.3	113.5	94.86	83.97	75.38	535.2	302.0	227.3	189.9	167.6	150.2
2	268.3	151.3	111.6	94.79	83.86	75.30	803.5	453.3	338.9	284.7	251.5	225.5
3	268.7	152.0	111.4	95.40	83.64	75.11	1072	605.3	450.3	380.1	335.1	300.6
4	268.4	152.1	111.5	95.35	83.49	75.31	1341	757.4	561.8	475.4	418.6	375.9
5	269.0	151.6	111.5	95.01	83.84	74.72	1610	909.0	673.4	570.4	502.4	450.6
6	268.4	152.4	111.4	96.02	83.97	75.07	1878	1061	784.7	666.5	586.4	525.7
7	268.2	151.4	111.9	98.27	84.33	75.02	2146	1213	896.6	764.7	670.7	600.7
8	268.1	151.8	111.5	98.56	84.27	75.41	2414	1365	1008	863.3	755.0	676.1
9	267.8	151.8	111.7	96.48	84.09	75.02	2682	1516	1120	959.8	839.1	751.2

Complete G0 falls from 267.9 s at p = 1 to 74.83 s at p = 6, and the ten groups from 2682 s to 751.2 s. With one worker the pool is 0.4% slower than sequential execution over all groups, which is the cost of moving 49 operand pairs and their products through the pool without any parallel gain. Small pairs are slower through the pool. Orders 16 to 64 run as one task, so at order 16 the pool needs 0.43 to 0.80 ms against 0.21 ms, and at order 64 every p takes 10.1 to 11.0 ms against 9.39 ms. At order 128 the seven tasks give 2.43 times speed-up at p = 6, but four, five and six workers all need two rounds, so their pair times stay between 35.0 and 39.0 ms. From order 256 the code creates 49 tasks, and every extra worker still helps. The 2048-order pair takes 81.03 s at p = 4, 71.29 s at p = 5 and 63.73 s at p = 6, so there is no flat step at p = 5. Within each p the group times differ by at most 4.0%, so the ordering of the process counts is stable.

Speed-up

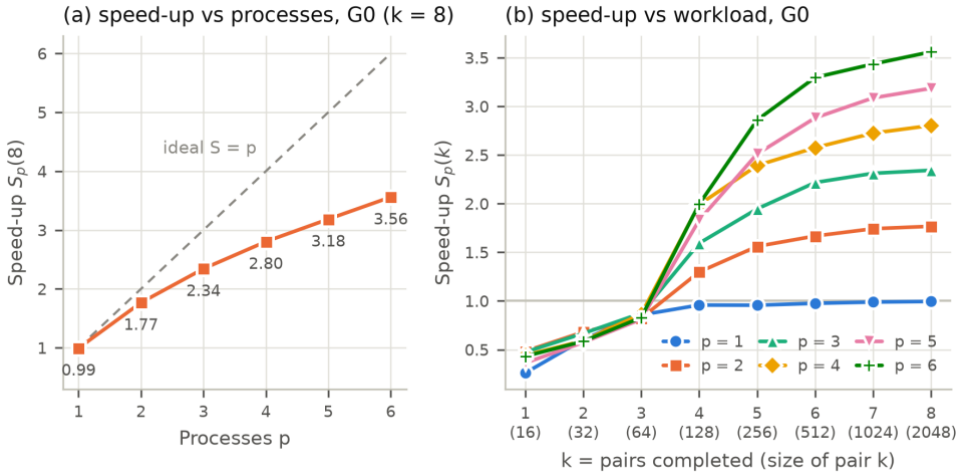


Figure D-2.1 Speed-up on G0: (a) versus process count for all eight pairs, (b) versus completed pairs k for each p

In Figure D-2.1(a) the speed-up of complete G0 rises from 0.994 at p = 1 to 3.56 at p = 6, below the ideal line but without a flat step. In Figure D-2.1(b) every curve starts below one, because orders up to 64 run as a single task, and p = 2 to 6 first exceed one at k = 4 (order 128). From k = 5, where 49 tasks are created, the curves separate in order of p and keep rising with k, because larger products carry more arithmetic for each transferred element. The six-worker curve rises from 2.86 at k = 5 to 3.56 at k = 8.

Table D-2.2 Speed-up from the cumulative times through each group

g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	0.994	1.767	2.343	2.803	3.185	3.560
1	0.998	1.768	2.350	2.812	3.186	3.555
2	1.006	1.783	2.385	2.839	3.214	3.585
3	1.002	1.775	2.386	2.826	3.206	3.574
4	0.999	1.769	2.385	2.818	3.201	3.564
5	0.998	1.767	2.385	2.815	3.196	3.564
6	0.997	1.764	2.387	2.810	3.194	3.562
7	0.997	1.764	2.386	2.797	3.189	3.561
8	0.997	1.763	2.387	2.787	3.187	3.558
9	0.996	1.762	2.386	2.784	3.185	3.558

Row 0 equals Sp(8) of G0, and each value changes by at most 2.0% as groups are added, so all ten groups confirm the G0 result. Over the full workload six workers reach 3.558, an efficiency of 59.3%.

Bottlenecks

Task rounds. With 49 equal tasks, p workers need ⌈49/p⌉ rounds. With the tail split, the last 49 mod p tasks become sevenths, so the rounds fall to ⌊49/p⌋ + ⌈7(49 mod p)/p⌉/7 and the bound at p = 6 rises from 5.44 to 5.91 (Table 6.2). The measured 3.558 is only 60.2% of that bound, so task rounds are not the main limit here.

Serial work and transfer. Per-task timestamps on the development machine of a 2048-order product at p = 6 show workers idle for about 2% of the pool phase, and main-process work before the first task and after the last one of 2.1% and 1.0% [9]. By Amdahl's law, a serial fraction of 3.1% alone would still allow about 5.2 times speed-up at p = 6 [6]. Data transfer is moderate, about 80 MB pickled to the workers and 64 MB back for that product.

Shared hardware. On the separate AMD Ryzen 7 PRO 7840U diagnostic machine the leaf kernel ran 1.84 times slower per multiply-add when six cores ran it together than on one busy core, because of the all-core clock limit and shared cache and memory traffic. That alone would limit six workers to about 6/1.84 = 3.3 on that machine [9]. This diagnostic does not quantify the corresponding slowdown on the i5-12500H, where the measured full-workload speed-up is 3.56. The i5-12500H has four performance cores, so additional workers may use efficiency cores or share a performance core; clock frequency may also change under load. The Windows records cannot isolate this effect, so the cause of the remaining gap is not established.

Small matrices. Up to order 64 the pool only adds transfer, and at order 128 seven tasks need two rounds on four to six workers. These pairs take less than 0.1 s, so they hardly affect the cumulative totals.

5.3 Cannon

Sequential results

Table B-3.1 Processing time of G0 under sequential implementation (s)

i	M	Cumulative T_i	Derived t_i
0	16	0.0003119	0.0003119
1	32	0.001693	0.001381
2	64	0.01126	0.009569
3	128	0.08103	0.06977
4	256	0.6434	0.5624
5	512	4.913	4.269
6	1024	38.49	33.57
7	2048	320.6	282.1

Table B-3.2 Sequential processing of all ten groups (s)

g	Group time $T(G_g)$	Cumulative through G_g
0	320.6	320.6
1	321.8	642.4
2	321.2	963.6
3	321.9	1286
4	321.5	1607
5	319.4	1926
6	317.7	2244
7	318.3	2562
8	317.7	2880
9	318.1	3198

The 2048-order pair takes 282.1 s, 88.0% of G0. From order 512 to 1024 and from 1024 to 2048 the pair time grows 7.9 and 8.4 times, close to the factor of eight of M^3 arithmetic. The ten groups take 317.7 to 321.9 s each, a spread of 1.3%, so G0 represents the whole workload.

Multiprocessing results

Table C-3.1 Processing time of G0 under multiprocessing implementation

	Measured cumulative MP time $T_i^{(p)}$						Derived MP time for the pair $t_i^{(p)}$					
i	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	0.001360	0.001590	0.001754	0.001821	0.001885	0.001861	0.001360	0.001590	0.001754	0.001821	0.001885	0.001861
1	0.003813	0.003124	0.003686	0.003731	0.003968	0.003739	0.002453	0.001534	0.001932	0.001910	0.002083	0.001877
2	0.01549	0.009396	0.009463	0.008203	0.007566	0.007070	0.01167	0.006272	0.005777	0.004472	0.003598	0.003331
3	0.08853	0.05570	0.05148	0.03680	0.03481	0.03247	0.07305	0.04630	0.04201	0.02860	0.02725	0.02540
4	0.6659	0.3785	0.3167	0.2326	0.2300	0.2090	0.5774	0.3228	0.2652	0.1958	0.1952	0.1765
5	5.023	2.809	2.312	1.720	1.765	1.532	4.357	2.431	1.995	1.487	1.535	1.323
6	39.38	21.80	17.95	13.42	13.77	11.70	34.35	18.99	15.64	11.70	12.00	10.17
7	320.1	181.3	147.7	113.6	113.9	96.97	280.7	159.5	129.8	100.2	100.2	85.27

Table C-3.2 Multiprocessing time of all ten groups (s)

	Measured MP time for the group $T^{(p)}(G_g)$						Cumulative MP time through the group					
g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	320.1	181.3	147.7	113.6	113.9	96.97	320.1	181.3	147.7	113.6	113.9	96.97
1	319.9	183.4	148.1	114.1	113.9	96.53	640.0	364.7	295.9	227.7	227.8	193.5
2	319.1	182.8	147.7	113.0	114.1	97.12	959.1	547.4	443.5	340.7	341.9	290.6

	Measured MP time for the group $T^{(p)}(G_g)$						Cumulative MP time through the group					
g	**p = 1**	**p = 2**	**p = 3**	**p = 4**	**p = 5**	**p = 6**	**p = 1**	**p = 2**	**p = 3**	**p = 4**	**p = 5**	**p = 6**
3	318.6	184.0	147.4	114.7	114.1	96.85	1278	731.4	590.9	455.4	456.0	387.5
4	318.2	181.7	148.1	112.6	113.6	98.00	1596	913.1	739.0	568.0	569.6	485.5
5	325.1	183.5	147.2	112.5	114.0	97.72	1921	1097	886.3	680.5	683.5	583.2
6	321.6	182.0	146.9	112.5	113.6	97.20	2243	1279	1033	792.9	797.1	680.4
7	320.7	182.4	146.9	112.4	113.5	97.10	2563	1461	1180	905.3	910.6	777.5
8	321.7	182.7	144.1	112.2	114.2	97.31	2885	1644	1324	1017	1025	874.8
9	327.9	183.0	143.8	112.5	112.9	98.12	3213	1827	1468	1130	1138	972.9

Two effects stand out in Tables C-3.1 and C-3.2. First, for orders 16 and 32 every p is slower than sequential execution: at order 16 the pool needs 1.4–1.9 ms against 0.31 ms, while from order 64 every $p \geq 2$ is faster. Second, five workers are no faster than four: the 2048-order pair takes 100.2 s with either, and the ten groups 1137.7 s against 1130.0 s, whereas six workers need 972.9 s. With one worker the pool is only 0.5% slower than sequential execution over all groups, and within each p the group times differ by at most 3.0%.

Speed-up

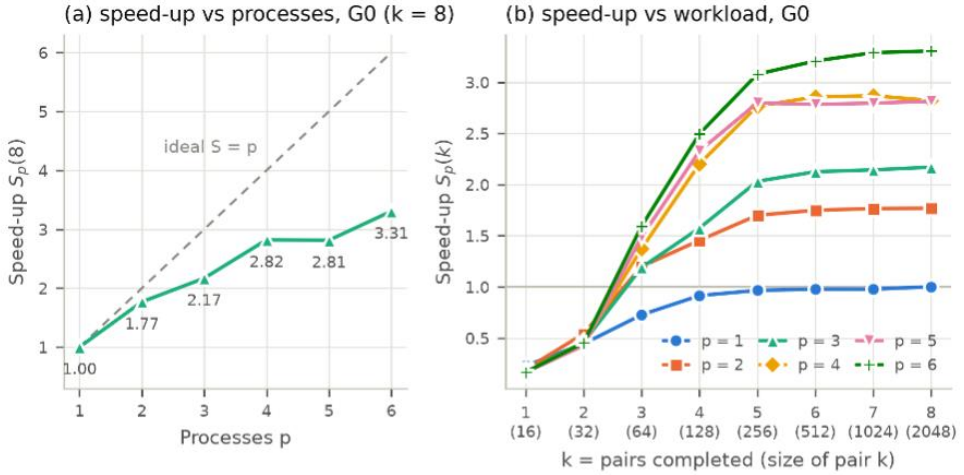


Figure D-3.1 Speed-up on G0: (a) versus process count for all eight pairs, (b) versus completed pairs k for each p

In Figure D-3.1(a) the speed-up for the complete G0 rises only to 3.31 at p = 6, far below the ideal line, and is flat from four to five workers. In Figure D-3.1(b) every curve starts below one at k = 1, crosses one at k = 3 (order 64) for $p \geq 2$, rises steeply until k = 5 (order 256) and then changes by less than 0.25, because the large pairs dominate the cumulative times.

Table D-3.2 Speed-up from the cumulative times through each group

g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	1.002	1.768	2.170	2.821	2.814	3.306
1	1.004	1.762	2.171	2.821	2.820	3.320
2	1.005	1.760	2.172	2.828	2.818	3.316
3	1.006	1.758	2.175	2.823	2.819	3.318
4	1.007	1.760	2.174	2.829	2.821	3.310

g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
5	1.003	1.757	2.174	2.831	2.818	3.303
6	1.001	1.755	2.172	2.830	2.815	3.298
7	1.000	1.754	2.171	2.830	2.814	3.296
8	0.998	1.752	2.175	2.831	2.810	3.292
9	0.995	1.751	2.179	2.830	2.811	3.287

Row 0 equals Sp(8) of G0, and each value changes by at most 1.2% as groups are added, so all ten groups confirm the G0 result. Over the full workload six workers reach 3.29, an efficiency of 54.8%.

Bottlenecks

Task rounds. The 16 grid tasks are equal, so p workers need ⌈16/p⌉ rounds and the speed-up cannot exceed 16/⌈16/p⌉: 2, 2.67, 4, 4 and 5.33 for p = 2 to 6 (Table 6.2). The measured speed-ups follow these steps. Four and five workers share the bound of 4 and reach 2.83 and 2.81; only the sixth worker, which cuts four rounds to three, raises the speed-up again. Per-task timestamps on the development machine, taken with an earlier build that used the same 16-task layout, agree: at order 1024 the workers are idle for 1.3% of the time at p = 4 but for 16.4% and 15.2% at p = 5 and 6, while building the tasks, transferring the blocks and joining Z take less than 1% of the run time [9].

Fixed cost per task. Every task is pickled, sent, scheduled and returned. At order 16 the pool adds 1.05–1.57 ms to the product, 3.4–5.0 times its whole sequential time, so the pool only pays off from order 64, where each block product carries enough arithmetic. For large matrices the cost is negligible, as the 0.5% difference at p = 1 shows.

Shared hardware. Task rounds do not explain why the measured speed-up falls from 88% of the bound at p = 2 to 62% at p = 6. On the development machine the block kernel took 12.3 ns per multiply-add on an otherwise idle machine, 15.2 ns with three other cores busy and 18.5 ns with five, as other cores became busy; shared power and clock limits are possible contributors, and additional workers may use slower efficiency cores [9]. The i5-12500H also has only four performance cores, so extra workers may use efficiency cores or share performance cores, depending on scheduling. The Windows records cannot isolate this effect, but it matches a loss that grows with the number of busy workers.

5.4 Fox

[B-4] Sequential Processing

Table B-4.1 Processing time of G0 under sequential implementation (s)

i	M	Cumulative T_i	Derived t_i
0	16	0.0003489	0.0003489
1	32	0.001973	0.001624
2	64	0.01294	0.01097
3	128	0.09196	0.07902
4	256	0.6609	0.5690
5	512	5.052	4.391
6	1024	39.37	34.32
7	2048	323.4	284.1

Table B-4.2 Sequential processing of all ten groups (s)

g	Group time $T(G_g)$	Cumulative through G_g
0	323.4	323.4
1	322.9	646.4
2	324.3	970.7
3	324.1	1295
4	323.7	1618
5	322.7	1941
6	323.8	2265
7	325.0	2590
8	327.0	2917
9	332.4	3249

Sequential Fox has $O(n^3)$ time complexity, so $T(2N)/T(N) \approx 2^3 = 8$. The measured ratio between consecutive pairs approaches 8 as N grows (~8.28 for pair 7). The workload is concentrated in the largest matrix pair. Pair 7 accounts for 87.83% of G0's total processing time. Thus, larger matrices offer greater potential for multiprocessing as their workload is higher.

[C-4] Multiprocessing

Table C-4.1 Processing time of G_0 under multiprocessing implementation

	Measured cumulative MP time $T_i^{(p)}$						Derived MP time for the pair $t_i^{(p)}$					
i	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	0.001025	0.0009399	0.0009589	0.001365	0.001405	0.001158	0.001025	0.0009399	0.0009589	0.001365	0.001405	0.001158
1	0.003291	0.002505	0.002131	0.002951	0.003206	0.002875	0.002266	0.001566	0.001172	0.001586	0.001801	0.001717
2	0.01511	0.009853	0.008368	0.007071	0.007443	0.006755	0.01182	0.007347	0.006238	0.004120	0.004238	0.003880
3	0.09901	0.05483	0.04711	0.04000	0.03614	0.03492	0.08389	0.04498	0.03874	0.03293	0.02870	0.02816
4	0.6892	0.4036	0.3197	0.2611	0.2602	0.2173	0.5902	0.3488	0.2726	0.2211	0.2241	0.1824
5	5.238	2.970	2.425	1.881	1.873	1.577	4.548	2.567	2.106	1.620	1.613	1.360
6	40.21	22.96	18.03	14.46	14.07	11.93	34.97	19.99	15.61	12.58	12.20	10.36
7	329.6	188.5	145.6	116.6	115.1	98.14	289.3	165.5	127.6	102.2	101.0	86.21

Table C-4.2 Multiprocessing time of all ten groups (s)

	Measured MP time for the group $T^{(p)}(G_g)$						Cumulative MP time through the group					
g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	329.6	188.5	145.6	116.6	115.1	98.14	329.6	188.5	145.6	116.6	115.1	98.14
1	329.3	186.9	147.6	116.3	116.1	104.0	658.9	375.4	293.2	232.9	231.2	202.2

	Measured MP time for the group $T^{(p)}(G_g)$						Cumulative MP time through the group					
g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
2	329.9	187.1	147.8	116.2	116.1	100.7	988.8	562.5	441.1	349.1	347.3	302.9
3	330.6	186.2	147.9	116.3	115.7	99.60	1319	748.7	589.0	465.4	463.0	402.5
4	330.7	184.6	147.0	116.3	115.7	99.04	1650	933.3	736.1	581.7	578.7	501.5
5	330.1	184.5	146.5	115.9	115.8	99.86	1980	1118	882.6	697.6	694.4	601.4
6	330.9	183.6	146.2	115.0	115.9	98.18	2311	1301	1029	812.6	810.3	699.6
7	330.7	183.5	146.5	114.8	118.7	98.84	2642	1485	1175	927.4	929.0	798.4
8	330.5	183.8	146.7	115.0	117.4	98.52	2972	1669	1322	1042	1046	897.0
9	335.4	182.1	150.2	114.6	116.9	98.71	3308	1851	1472	1157	1163	995.7

Processing time decreases as the number of workers increases, but the reduction is not proportional to the worker count. Increasing from 4 to 5 workers reduces G_0's total time by only 1.29% as 16 tasks cannot be evenly distributed among 5 workers. With 6 workers, each handle at most three tasks, achieving the shortest measured time. Pair 7 accounts for 87.84% of the six-worker total, so its performance largely determines overall acceleration.

[D-4] Determine the speed-up

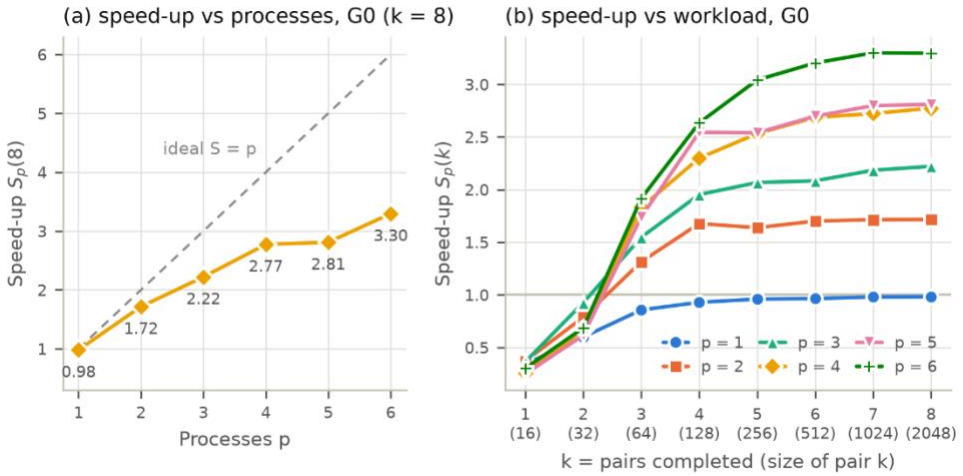


Figure D-4.1 Speed-up on G0: (a) versus process count for all eight pairs, (b) versus completed pairs k for each p

As seen in Figure D-4.1(a), speed-up increases with the number of workers, achieving 3.30× with 6 workers, below the ideal linear speed-up of 6×. With 16 independent tasks, 6 workers require at least 3 rounds giving an ideal ceiling of 16/3 ≈ 5.33x speed-up for ideal equal-duration tasks. Amdahl's law further explains how sequential block preparation and final assembly constrain overall speed-up. Serialisation and inter-process transfer of 8 input blocks per task introduce additional overhead. Figure D-4.1(b) shows that smaller matrices (16 and 32) take longer under multiprocessing than sequential execution. For 2 or more workers, speed-up exceeds 1× after the third pair (N = 64), then increases before levelling off near 3.30× with six workers. Larger computations therefore help offset overhead, but acceleration remains constrained by sequential work, task distribution and communication costs.

Table D-4.2 Speed-up from cumulative times through each group

g	p = 1	p = 2	p = 3	p = 4	p = 5	p = 6
0	0.981	1.716	2.221	2.773	2.810	3.296
1	0.981	1.722	2.204	2.775	2.796	3.197
2	0.982	1.726	2.201	2.781	2.795	3.205
3	0.981	1.729	2.198	2.782	2.797	3.217
4	0.981	1.734	2.199	2.782	2.797	3.227
5	0.980	1.737	2.199	2.783	2.795	3.228
6	0.980	1.740	2.202	2.787	2.795	3.238
7	0.980	1.744	2.204	2.793	2.788	3.244
8	0.981	1.748	2.207	2.798	2.788	3.252
9	0.982	1.756	2.207	2.808	2.793	3.264

The cumulative speed-up across all ten groups closely follows the trend observed for G0. At six workers, it reaches 3.26×, compared with 3.30× for G0 alone, showing a similar overall benefit from multiprocessing across the full workload.

Bottlenecks

Task imbalance. The implementation splits the work into 16 equal tasks for at most 6 workers. A separate diagnostic run at N = 2048 on Windows 11 with an AMD Ryzen 7 5800H and 16 GB RAM demonstrated this imbalance [9]. Four workers received four tasks each and completed parallel execution in 101.80 s. Five workers received 4, 3, 3, 3 and 3 tasks. The maximum task count was unchanged, so execution time improved only slightly to 100.85 s, while the four lighter workers finished at about 77.55–78.26 s and remained idle for approximately 23 s. Six workers reduced the maximum task count to three and execution time to 78.19 s, but two workers finished at about 52.91–53.07 s and remained idle for approximately 25 s. Fox et al.'s algorithm [11] avoids this by assigning one output block to each of $p = q^2$ processors, which requires 16 processors for q = 4.

Overhead. In the six-worker diagnostic, sequential preparation and assembly took only 0.78 s of the 78.97 s total measured time, approximately 1%, so these stages were not the main bottleneck at N = 2048. Passing blocks between processes also adds serialisation and communication costs, although these were not measured separately in this diagnostic.

Shared hardware. Task imbalance alone does not explain the gap between the ideal 5.33× ceiling and the main benchmark's measured 3.30× speed-up. In the separate diagnostic at N = 2048, average task-computation time increased from 23.57 s with one worker to 25.94 s with six workers, approximately 10%. This increase possibly reflects increased competition for shared hardware resources, such as CPU cache and memory bandwidth, as more workers execute simultaneously. The resulting slowdown could therefore partly explain why the lower measured speed-up. However, the diagnostic used a different machine, so the exact cause of the main benchmark's speed-up shortfall cannot be fully supported by the diagnostic.

6 Joint discussion

Sequential cost

Table 6.1 Execution time on the i5-12500H (s): sequential time of each G_0 pair and total time over all groups

Workload	Strassen	CW	Cannon	Fox
G_0 pair, M = 16	**0.0002006**	0.0002080	0.0003119	0.0003489
G_0 pair, M = 32	**0.001184**	0.001199	0.001381	0.001624
G_0 pair, M = 64	0.01030	**0.009390**	0.009569	0.01097
G_0 pair, M = 128	0.08560	0.08496	**0.06977**	0.07902
G_0 pair, M = 256	0.6438	0.6422	**0.5624**	0.5690
G_0 pair, M = 512	4.549	4.656	**4.269**	4.391
G_0 pair, M = 1024	**31.85**	32.74	33.57	34.32
G_0 pair, M = 2048	**224.8**	228.3	282.1	284.1
G_0–G_9, sequential	**2639**	2672	3198	3249
G_0–G_9, p = 6	771.1	**751.2**	972.9	995.7

Table 6.1 lists the sequential time of each G0 pair and the totals over all groups. Up to order 64, Strassen and CW do not split, because their leaf size is 64, and run the plain triple loop. Cannon and Fox split every order into 16 blocks and join the output, so at order 16 they take 1.56 and 1.74 times as long as Strassen. From order 128, Strassen and CW replace eight half-size products by seven at every level, so a 2048-order product needs $7^5 \times 64^3 = 4.41$ billion scalar multiplications instead of 8.59 billion. The measured growth follows these counts: from order 1024 to 2048, runtime increases by a factor of 7.06 for Strassen and 6.97 for CW, compared with 8.40 for Cannon and 8.28 for Fox.

However, the savings require additional block additions and new lists. Strassen and CW spend 35.8–39.3 ns per scalar multiplication at order 64 and 51.0–51.8 ns at order 2048, compared with 31.3–33.9 ns for Cannon and Fox from order 256 onwards. This measure includes additions, allocation and other overhead; it is not the time of an isolated multiplication. The timings also reflect implementation differences: Strassen and CW use zip-based inner loops, whereas Cannon and Fox use indexed loops. Strassen and CW are faster than Cannon at orders 16 and 32, slower from order 128 to 512, by 22–23% at order 128 and 7–9% at order 512, and faster again from order 1024, by 19–20% at order 2048.

The 2048-order pair accounts for 85.7–88.0% of a group's runtime and therefore dominates the totals: 2639.4 s for Strassen, 2672.3 s for CW, 3198.2 s for Cannon and 3249.4 s for Fox. Although CW requires 15 block additions per recursive level rather than Strassen's 18, its total runtime is 1.2% higher. This difference is small relative to CW's observed 3.3% group-to-group spread, although that comparison alone does not establish statistical equivalence. Cannon and Fox compute the same set of block products, and their total runtimes differ by 1.6%.

Speed-up with multiprocessing

The implementations hand out different units of work. With $p \geq 2$, Strassen and CW expand the top recursion levels in the main process and send the products as tasks: one task up to order 64, seven at order 128 and 49 from order 256, with CW additionally splitting the last 49 mod p tasks into seven smaller ones from order 512. Cannon and Fox send one task per output block,

16 equal tasks at every order. With n equal tasks, p workers need ⌈n/p⌉ rounds, which bounds the speed-up by n/⌈n/p⌉. Table 6.2 lists the ideal task-round bounds for the 2048-order products.

Table 6.2 Ideal task-round speed-up bounds for order 2048

	p=1	p=2	p=3	p=4	p=5	p=6
Bound Strassen	1.000	1.960	2.882	3.769	4.900	5.444
Bound CW	1.000	1.994	2.983	3.988	4.971	5.914
Bound Cannon, Fox	1.000	2.000	2.667	4.000	4.000	5.333

The measured speed-ups follow the steps of the bounds. Up to p = 4 the four algorithms stay within 0.21 of each other. From p = 4 to 5, Cannon and Fox stay at 2.81 and 2.79, because 16 tasks still need four rounds, while Strassen and CW rise to 3.15 and 3.18 with 49 tasks. At p = 6, CW reaches 3.56, Strassen 3.42, Cannon 3.29 and Fox 3.26. CW has the highest bound at p = 6, 5.91 against 5.44 for Strassen: 49 = 8 × 6 + 1 leaves one task for a ninth round, and CW splits it into seven smaller tasks.

Relative to its bound, every algorithm keeps a similar share at p = 2, 3, 4 and 6: 87.5–88.4%, 80.0–82.8%, 69.8–72.6% and 60.2–62.9%. At p = 5 Cannon and Fox keep 69.8–70.3%, as their fifth worker mostly waits, against 64.1–64.3% for Strassen and CW. Task rounds thus explain the differences between the algorithms, while the remaining loss grows with the number of busy workers, even though only Strassen and CW add blocks in the main process. At p = 1 the pool costs 0.4–1.8% more time than sequential execution, and at p = 6 the efficiency is 54.4–59.3%.

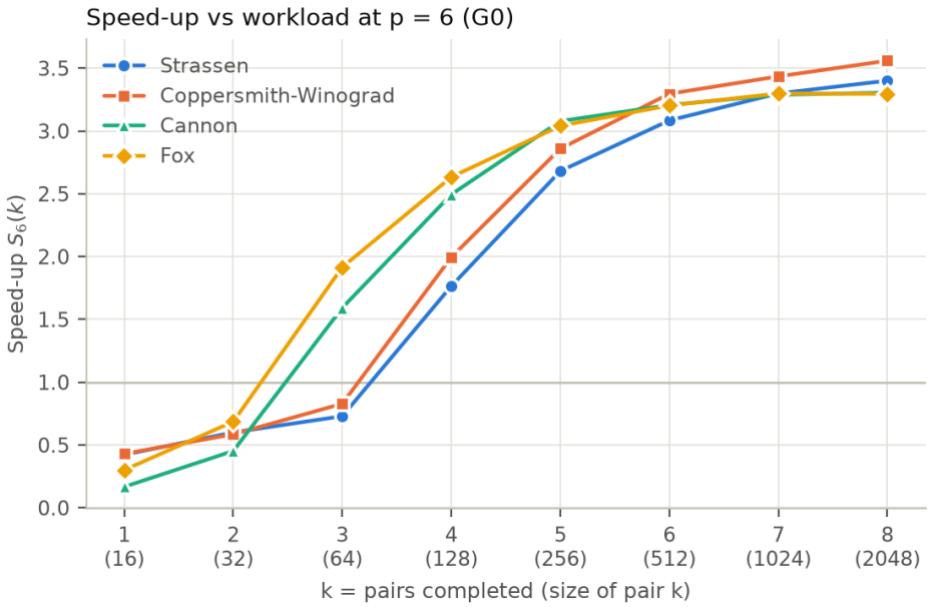


Figure 6.1 Cumulative speed-up at p = 6 versus completed pairs of G0

Figure 6.1 shows how the six-worker speed-up builds up over the G0 pairs. At order 16 every algorithm is slower through the pool for every p, with pair speed-ups of 0.17–0.48. Strassen and CW send a product of order 64 as a single task and gain nothing at that order (0.75–1.03), whereas Cannon and Fox already reach 1.49–2.87 there for p ≥ 2. At order 128, seven tasks give Strassen and CW pair speed-ups of 2.19 and 2.43 at p = 6, against 2.75 and 2.81 for Cannon

and Fox. From order 1024 the 49-task algorithms lead, and at order 2048 the p = 6 pair speed-ups are 3.42, 3.58, 3.31 and 3.30. After k = 6 the cumulative speed-ups rise by at most 0.32. With six workers the ten groups take 751.2 s with CW, 771.1 s with Strassen, 972.9 s with Cannon and 995.7 s with Fox.

Strengths and pitfalls

Strassen and CW are the only algorithms whose arithmetic grows more slowly than M^3, which gives them the lowest sequential and six-worker totals. Their pitfalls come from the same recursion. Strassen's analysis counts arithmetic operations [2]; it does not model Python lists, allocation or the size of intermediate integers. The measured cost per scalar multiplication rises by 30–45% from order 64 to 2048. The large-matrix advantage over the tested block implementations begins at order 1024. The recursion needs orders that halve down to the leaf: the assigned powers of two do, and other orders stop at the first odd size and fall back to the triple loop. Parallel work exists only in the recursion tree, so products up to order 64 run as one task, order 128 as seven, and 49 tasks leave workers idle in the last round unless they are split further, as in CW. The three additions per level that the Winograd form saves do not appear in the measured times. The construction of Coppersmith and Winograd [3] is an asymptotic result, and the implementation follows the seven-product form of Appendix A instead [1].

Cannon and Fox keep the M^3 arithmetic of the conventional product and take 1.25–1.26 times as long as Strassen at order 2048. In exchange they divide the work into 16 equal and independent tasks at every assigned order, which gives the earliest parallel gain, from order 64, and the highest six-worker pair speed-ups up to order 256. Their pitfall is the fixed 4 × 4 grid of the revised statement: 16 tasks cap the speed-up at $16/\lceil 16/p \rceil$, so a fifth worker adds nothing and six workers are bounded by 5.33. The original formulations use q^2 processors with local blocks and neighbour transfers (Cannon [10]) or broadcasts (Fox [11]). Our pool has at most six workers and no shared memory. Each task therefore receives its whole X block row and Y block column, with each block copied four times. Shifting versus broadcasting no longer changes communication between workers. Accordingly, the speed-ups of Cannon and Fox differ by at most 0.03 at every p.

Every pool run at order 16 is 2.1–6.0 times slower than sequential execution: passing tasks outweighs its 0.20–0.35 ms arithmetic. The algorithms' operation counts omit this fixed cost.

Development machine

On the Apple M5 used for development [9], the same code and data keep the sequential order: 1289.6 s for Strassen, 1358.5 s for CW, 1741.1 s for Cannon and 1746.0 s for Fox, 49–54% of the Windows times. The six-worker speed-ups are lower for all four, 2.28, 2.44, 2.37 and 2.52, and Cannon and Fox again gain nothing from p = 4 to 5 (2.12 to 2.08 and 2.08 to 2.09). Cannon and Fox, which run the same 16-task scheme, differ by 0.14 at p = 6 there, so speed-up differences of this size between algorithms depend on the platform, whereas the sequential order and the task-round steps hold on both machines.

7 Conclusion

Multiprocessing helps once arithmetic outweighs overhead: Cannon and Fox gain from order 64, and Strassen and Coppersmith–Winograd from order 128. Their six-worker speed-ups across ten groups are 3.29, 3.26, 3.42 and 3.56, respectively. Sixteen grid tasks make a fifth worker ineffective; recursive task splitting allows further gains up to six. Task rounds, serial work and transfer limit scaling. Speed-up grows mainly up to order 512. From order 1024, reduced arithmetic gives Strassen and Coppersmith–Winograd the shortest sequential and six-worker times.

8 Intuitive implementation walkthrough

8.1 Strassen

[F-1] Intuitive Implementation Walkthrough – ZHOU JUNMIN

Imagine cutting each matrix into four square tiles. The usual tile method needs eight multiplications to produce the four answer tiles. Strassen instead prepares seven special pairs by adding or subtracting input tiles. After multiplying those pairs, it combines the seven answers so that the extra terms cancel and the four correct output tiles remain. The same recipe repeats inside a tile until its side is 64 or smaller. At that point, my code calculates each answer by walking along one input row and one input column. The saved multiplication is useful because it is exchanged for simpler additions, although those additions still take time [1], [2].

The parallel version opens the recursion tree before handing out work. For a 128-order matrix, it creates seven jobs. From order 256 upwards, with two to six workers, it opens two levels and creates 49 jobs. The pool then behaves like a shared queue: a free worker takes the next pair, completes its smaller recursive calculation and returns an answer tile. The main process collects each family of seven answers and rebuilds the matrix. Workers do not create more pools. With one worker, or a matrix at or below the cutoff, one whole job is still sent through the pool.

More workers do not divide the whole job equally. The main process still converts arrays to lists, prepares tiles and joins answers. Tiles must also be packed, copied to another process and copied back. Forty-nine jobs do not divide evenly among six workers: after eight full rounds, one job remains while other workers can be idle. These costs explain why small matrices can be slower through the pool and why six workers produce about 3.42 times speed-up over all ten groups, rather than six times.

The timer starts only after every worker is ready, so pool creation is not responsible for the measured small-workload slowdown. The remaining costs occur while the calculation runs. More detailed timing would be needed to separate communication, scheduling and memory effects. The seven-product mechanism follows Appendix A and Strassen's method [1], [2]; the importable worker functions and spawn-based Pool setup follow Python's multiprocessing model [5].

8.2 Coppersmith-Winograd

[F-2] Intuitive Implementation Walkthrough – RICARDO JACK LIWONGAN

A normal matrix product pairs every row of the first matrix with every column of the second, which means billions of small multiplications for a 2048 by 2048 matrix. My code first cuts each matrix into four equal quarters. Done the plain way, rebuilding the answer from quarters needs eight quarter-sized multiplications. Winograd's version first builds a few new quarters by adding and subtracting the original ones, and with those it needs only seven. The four answer quarters are then rebuilt by adding the seven results in fixed combinations, and one shared partial sum is reused by three of them, so the whole step needs only 15 additions. Additions are much cheaper than multiplications, so trading one multiplication for a few additions saves time. It is like a baker who must fill four large cake orders. Instead of baking eight cakes, the baker first mixes a few combined batters, bakes only seven cakes and then cuts and recombines them into the four orders. Mixing is quick but every bake is slow, so skipping one bake is a real saving. The same trick is applied to each of the seven smaller multiplications, and again to theirs, until a block is 64 by 64. Below that size the extra additions cost more than they save, so the code multiplies the block with a plain row by column loop. For a 1024 by 1024 matrix this gives 2401 small block multiplications instead of 4096, about 41% fewer.

The parallel version lets the main process do the first two rounds of cutting and adding. This turns one large multiplication into 49 smaller ones that are completely independent, for example 512 by 512 each for a 2048 by 2048 matrix. It works like a kitchen where the head chef prepares 49 trays, each free cook takes the next tray, cooks it with the same recipe as the sequential version and hands the finished dish back. Cooks never pass food to each other. With six workers, 49 jobs do not divide evenly, so one job would run alone at the end while five workers wait. To avoid this, the main process cuts the leftover job into seven smaller pieces, so all workers share the last round. As each group of seven results is collected in order, the main process adds them into a larger block while the workers are still computing and finally assembles the full answer.

More workers do not divide the time equally. The main process still converts the arrays to lists, does the first additions and joins the final answer, and no worker can help with that. Every input block and every result is copied between processes, because they do not share memory. For matrices up to 64 by 64 there is only one job, so the copying only adds cost and the pool is slower than the plain loop. Starting the worker processes takes another 0.27 to 0.41 s, but that happens before the timer starts, so it is not part of the measured times. Another effect is that busy cores slow each other down. On the separate diagnostic laptop, each core ran the multiplication loop 1.84 times slower when six cores ran it together, which may reflect shared power limits, cache and memory traffic. These are possible contributors to why six workers give about 3.56 times speed up over all ten groups rather than six times.

8.3 Cannon

[F-3] Intuitive Implementation Walkthrough – ZHANG FANGYU

Picture sixteen people sitting in a 4 × 4 square, each in charge of one tile of the answer. Everyone starts with one tile of X and one tile of Y. Before the work begins, everyone in row i passes their X tile i seats to the left and everyone in column j passes their Y tile j seats up, so each person now holds two tiles that belong together. Four rounds follow. In each round everybody multiplies the two tiles in their hands, adds the result to their own answer tile, then passes the X tile one seat left and the Y tile one seat up. After four rounds every person has met exactly the four tile pairs their answer needs, and nobody ever waited for a tile someone else was still using. Multiplying two tiles is ordinary long multiplication: walk along a row of one tile and a column of the other, multiply matching numbers and add them up [1].

My computer has at most six workers, not sixteen people, so I turn each seat into a job. Instead of passing tiles around, the main process hands each job the four tile pairs its seat would receive, in round order. The pool works like a queue at a counter: whichever worker is free takes the next job, does the four multiplications and returns one finished answer tile. The main process then glues the sixteen tiles back into the answer matrix. Because each job already has everything it needs, workers never talk to each other or wait for a round to finish.

Six workers make the program about 3.3 times faster, not six times, for three reasons.

First, sixteen equal jobs do not split evenly. Five workers still need four rounds, the same as four workers, so a fifth worker adds almost nothing. Six workers need three rounds, with two of them idle in the last one.

Second, the processes do not share memory, so every tile is copied into a worker and every answer tile is copied back. For tiny matrices this postage costs more than the arithmetic: a 16 × 16 product takes four to six times longer through the pool.

Third, busy cores slow each other down. On my Mac testing, one core ran about 1.5 times slower while five others were busy, and with only four fast cores, extra workers may use slower cores, depending on scheduling.

To make development and testing more convenient, I built the test bench the whole team shares. One settings file holds all the common rules: the random seed that gives everyone identical matrices, the matrix sizes, the process counts from one to six, and the timing rule. One timer serves all four algorithms. It starts only after every worker is ready, so starting the pool is never counted, and it records the running total after each matrix pair. A checker compares each algorithm's answers with NumPy's before any timing run. A single script then runs all four algorithms in turn, lets the laptop cool down in between and draws the tables and figures. Because of this, the differences in our results come from the algorithms, not from different data or different stopwatches.

8.4 Fox

[F-4] Intuitive Implementation Walkthrough – DIVYALAKSHMI BALASUBRAMANIAM

Think of Fox's algorithm as assembling a large picture out of smaller tiles. The picture is the output, Z. To build one tile of Z, take the matching row of tiles from X and the matching column of tiles from Y and line them up in pairs. That gives four pairs. Multiply each pair, then add the four results to get that finished Z tile. Fox's algorithm is a schedule that tells every tile of Z which pair to work on in each of 4 stages, so no pair is skipped or repeated. My code cuts X and Y into a fixed 4 × 4 grid of tiles, giving 16 tiles each [1]. The grid is always 4 × 4, so bigger matrices simply get bigger tiles, not more of them.

Each stage has 3 steps. First, we pick and share Y. We take one diagonal of Y's tiles ($Y_{0,0}$, $Y_{1,1}$, $Y_{2,2}$, $Y_{3,3}$) and copy each of those tiles down its column, so every row of the grid has the Y tile it needs. Next, each X tile is multiplied by the Y tile it received, and the result is added to the running total for the matching tile of Z. Lastly, we slide X's tiles one position to the left, with the tile that falls off the edge wrapping around to the end, and move to the next diagonal of Y ($Y_{1,0}$, $Y_{2,1}$, $Y_{3,2}$, $Y_{0,3}$). After 4 stages, X's tiles have been slid left 3 times, and every diagonal of Y has been selected exactly once. By then, every tile of Z has received all four of its pairs, so the picture is complete. This is Fox's broadcast-and-roll approach [11].

In Fox's algorithm [11], broadcast and roll physically send data between processors. In my Python version, nothing physically moves. Broadcasting means reusing the same tile from a list wherever it's needed and rolling means shifting which list position I read from. I simulate both by changing which tiles I look at.

To speedup this process, I can split the work by tile. With a 4 × 4 grid, the picture has 16 tiles, so there are 16 independent tasks, and each task is to build one tile of Z. A worker receives the matching row of X tiles and column of Y tiles, runs all 4 stages on its own, and returns the finished tile together with its position. The main process collects the 16 returned tiles and places them in their positions to form Z. It never has to add up partial results from different workers, because every task owns a different tile of Z. One worker can complete several tasks, so with more workers, more tiles are built at the same time. This is multiprocessing.

Adding workers does not make the multiplication proportionally faster because not all the time is spent multiplying. Some steps stay sequential. The main process converts the matrices into lists, cuts them into tiles, prepares the tasks and assembles the final picture. No worker can help with these steps, and they set a ceiling on the best possible speed-up. Passing tasks also has a cost. Python must package each task's tiles, transfer them to a worker and rebuild them there, then send the finished tile back. The same tiles are sent repeatedly because every task carries 4 tiles of X and 4 tiles of Y. Across 16 tasks, that is 128 tiles, even though only 32 unique tiles exist. This means the same data is transferred multiple times, increasing communication overhead.

The 16 tasks also cannot always be shared equally among workers. For example, with 4 workers, the tasks can be divided evenly, with 4 tasks per worker. With 6 workers, the tasks cannot be divided evenly, so some workers may have more work than others. Workers that finish early may have to wait for the remaining workers, limiting the speed-up. In my code, Python decides how to distribute the tasks, so the actual workload may vary. Finally, workers also compete for shared computer resources, such as memory bandwidth and CPU cache, which can reduce performance as more workers are added.

9 References

[1] CEG5201 Hardware Technologies, Principles, & Platforms, CA2 Revised Assignment Statement, Semester I, AY2026/2027, released 27 September 2026. Sections I-V and Appendix A. Supplied course document.

[2] V. Strassen, "Gaussian elimination is not optimal," Numerische Mathematik, vol. 13, no. 4, pp. 354-356, 1969, doi:10.1007/BF02165411.

[3] D. Coppersmith and S. Winograd, "Matrix multiplication via arithmetic progressions," Journal of Symbolic Computation, vol. 9, no. 3, pp. 251-280, 1990, doi:10.1016/S0747-7171(08)80013-2.

[4] S. Winograd, "On multiplication of 2 × 2 matrices," Linear Algebra and its Applications, vol. 4, no. 4, pp. 381-388, 1971, doi:10.1016/0024-3795(71)90009-7.

[5] Python Software Foundation, "multiprocessing - Process-based parallelism," Python 3.14 documentation. https://docs.python.org/3.14/library/multiprocessing.html. Accessed 25 October 2026.

[6] V. Bharadwaj, CEG5201 Chapter 1, July 2026, PDF p. 75, Amdahl's law and speed-up. Supplied lecture slides.

[7] V. Bharadwaj, CEG5201 Chapter 2, July 2026, PDF pp. 14-17, 22-28, 36-46 and 68-71, independence, granularity, communication and multicore memory. Supplied lecture slides.

[8] NumPy developers, "numpy.savez" and "Random Generator," NumPy documentation. https://numpy.org/doc/stable/reference/generated/numpy.savez.html and https://numpy.org/doc/stable/reference/random/generator.html. Accessed 25 October 2026.

[9] Group 4, CEG5201 CA2 group repository, CEG5201_CA2. Source: algorithms/ and common/; common benchmark: results_windows/*/raw, tables and figures; Apple M5 runs: results_mac_M5; AMD Ryzen 7 PRO 7840U CW diagnostics: results_windows/coppersmith_winograd/profile/mp_diagnosis.txt; Fox diagnostics on a separate Ryzen 7 5800H: fox_diagnostic/fox_diagnostic_output.txt. Internal project records.

[10] L. E. Cannon, "A Cellular Computer to Implement the Kalman Filter Algorithm," PhD thesis, Montana State University, Bozeman, MT, USA, 1969.

[11] G. C. Fox, S. W. Otto and A. J. G. Hey, "Matrix algorithms on a hypercube I: Matrix multiplication," Parallel Computing, vol. 4, no. 1, pp. 17–31, 1987, doi:10.1016/0167-8191(87)90060-3.